Synapse Analyzer

OAuth 2.0, Cerberus SSO integration & authentication architecture

Technical analysis — covering the authorization code flow with PKCE, the Cerberus endpoint changes, the OpenID discovery surface, browser-side authentication, and cross-origin configuration across both applications.

Contents

1. **System context and the trust relationship**
2. **Why authorization code + PKCE**
    1. Public versus confidential clients
    2. What PKCE actually defends against
3. **State of the implementation before the changes**
4. **The complete end-to-end flow**
5. **Endpoint-by-endpoint changes**
    1. Parameter binding at the controller
    2. The authorize endpoint
    3. The token endpoint
    4. Client registration
    5. The login and MFA endpoints
6. **The WellKnownController and token validation**
7. **Browser-side authentication in the React client**
8. **The Angular application as authorization UI**
9. **Cross-origin resource sharing**
10. **Other logic worth noting**
11. **Security posture and remaining gaps**
12. **Configuration reference**

1. System context and the trust relationship

Four independently deployable pieces participate in authentication. Understanding which one holds which secret — and which one is merely a courier — is the key to everything that follows.

Component	Port	Role in authentication
Synapse client React SPA	5173	The *relying party*. Starts the flow, holds the PKCE verifier, exchanges the code, stores the token. Holds no secret.
Synapse API ASP.NET Core	5056	The *resource server*. Validates tokens; never issues them and never sees a credential.
Cerberus API ASP.NET Core	5211	The *authorization server*. Owns users, issues codes and tokens, publishes the signing key.
Cerberus web app Angular	4200	The *authorization UI*. The only place a password is ever typed.

The property that makes this SSO rather than a shared login page is that **the Synapse client never sees the user's credentials**. It hands the browser to Cerberus, and Cerberus hands back a short-lived, signed assertion. The Synapse API then verifies that assertion cryptographically without ever contacting the Synapse client, and without sharing a secret with Cerberus.

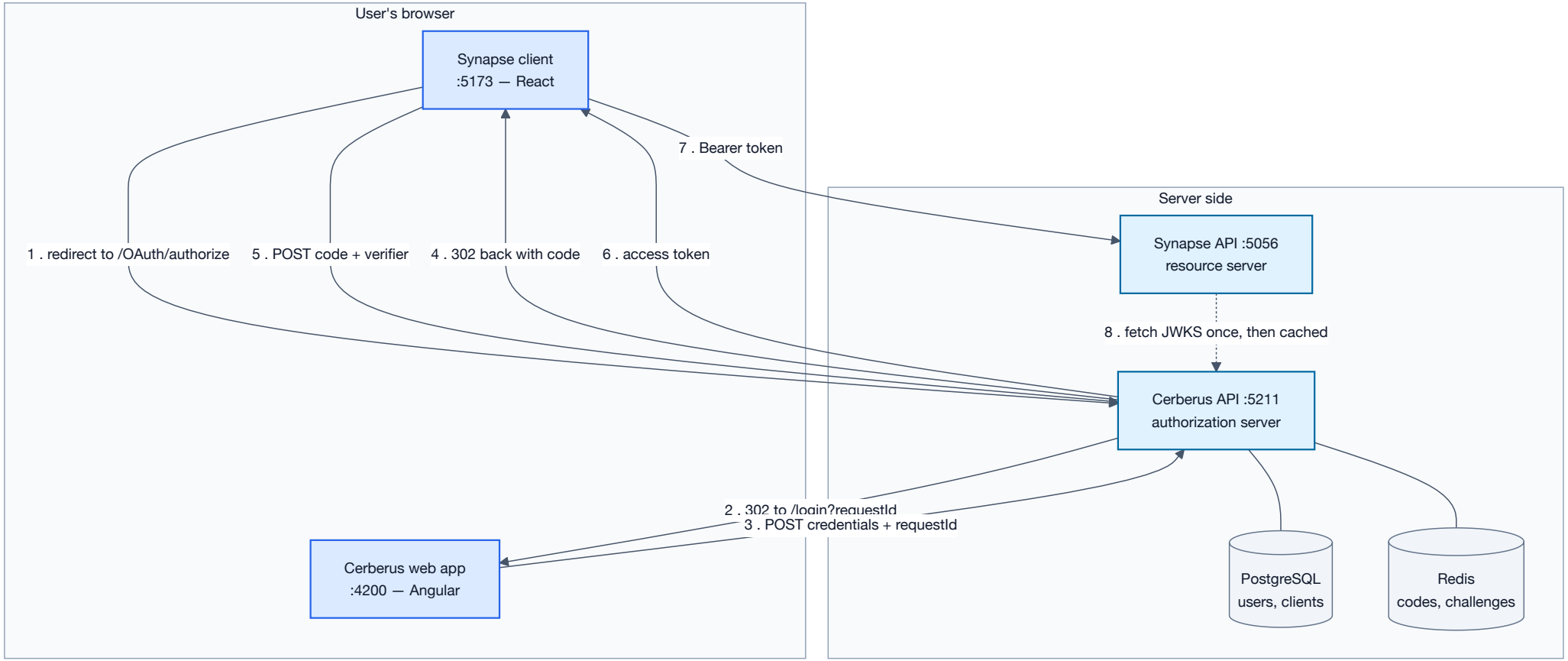

Figure 1 — Component context. Step 8 is the trust anchor: the resource server validates tokens against the authorization server's published public key.

2. Why authorization code + PKCE

2.1 Public versus confidential clients

OAuth divides clients into two kinds, and the distinction drives every design decision here.

- A **confidential client** runs on a server and can keep a `client_secret`. It proves its identity at the token endpoint by presenting that secret.
- A **public client** runs where the user can read its code — a browser SPA, a mobile app. It *cannot* keep a secret. Anything shipped to the browser is readable by anyone who opens developer tools.

The Synapse client is unambiguously a public client. This matters because the original Cerberus token endpoint *required* `client_secret` unconditionally, which meant a browser SPA could only complete the flow by embedding a secret in downloadable JavaScript. That is not a weaker configuration; it is the same as having no authentication on the token endpoint at all, because the "secret" is public.

The implementation now decides which rule applies from **how the client is registered**, never from what the caller sends:

```
var isPublicClient = string.IsNullOrEmpty(client.ClientSecret);
```

> **DESIGN NOTE**
> Deriving the client type from server-side registration rather than from the request is deliberate. If the branch were chosen by "did the caller send a secret?", an attacker holding a stolen authorization code could simply omit the secret and take the PKCE-only path, or omit the verifier and take the secret path — downgrading to whichever check they could satisfy.

2.2 What PKCE actually defends against

Proof Key for Code Exchange (RFC 7636) closes the *authorization code interception* attack. The authorization code travels back to the client through a browser redirect — through the URL bar, through browser history, potentially through a referrer header or a malicious app registered for the same redirect scheme. If possession of the code were sufficient to obtain a token, intercepting that redirect would be enough to impersonate the user.

PKCE binds the code to the specific client instance that requested it:

1. Before redirecting, the client generates a high-entropy random **verifier** and keeps it locally.
2. It sends only `SHA-256(verifier)` — the **challenge** — with the authorization request.
3. Cerberus stores that challenge alongside the issued code.
4. At the token endpoint the client must present the original verifier. Cerberus hashes it and compares.

Because SHA-256 is one-way, an attacker who intercepts the redirect obtains the code and the challenge but cannot derive the verifier. The code alone is worthless.

The implementation lives in `Application/CommandsAndQueries/Clients/Pkce.cs`:

```
public static bool Verify(string? codeVerifier, string? expectedChallenge)
{
    if (string.IsNullOrWhiteSpace(codeVerifier) ||
        string.IsNullOrWhiteSpace(expectedChallenge))
        return false;

    // RFC 7636 section 4.1
    if (codeVerifier.Length is < 43 or > 128)
        return false;

    var actual = ComputeChallenge(codeVerifier);

    return CryptographicOperations.FixedTimeEquals(
        Encoding.ASCII.GetBytes(actual),
        Encoding.ASCII.GetBytes(expectedChallenge));
}
```

Three details are load-bearing:

- **The length bounds are a security check, not validation politeness.** RFC 7636 mandates 43–128 characters. Accepting a three-character verifier would make the challenge brute-forceable and silently defeat the mechanism.
- **Fixed-time comparison.** The challenge is a secret-derived value. An ordinary string comparison returns early on the first differing byte, and the timing difference leaks the expected value one byte at a time.
- **Only `S256` is accepted.** The spec also permits `plain`, where the challenge *is* the verifier — which offers no protection at all against an attacker who intercepted the authorization request. The authorize endpoint rejects any other method.

3. State of the implementation before the changes

The flow could not complete. Not "was insecure" — could not complete. Six independent defects each individually blocked it; they are listed in the order a request would encounter them.

#	Defect	Effect
1	OAuth sends `client_id`, `response_type`, `code_challenge`; ASP.NET model binding matches on *property name* (`ClientId`, `ResponseType`).	The DTO arrived entirely empty. Every authorization failed at the first check with "*Unsupported response_type*", regardless of what was sent.
2	No `Client` table. The entity existed in the EF model but no migration ever created it.	No OAuth client could be registered, so no client could ever be valid.
3	`/OAuth/authorize` redirected to the relative path `/api/auth/login?requestId=…`.	Resolved against the API's own origin, where no such route exists. The flow died at its first redirect with a 404.
4	PKCE was accepted from the client but never implemented server-side. No `code_challenge` field existed on the request DTO and no `code_verifier` on the token DTO.	PKCE parameters were silently discarded. The token endpoint required a `client_secret` a browser cannot hold.
5	`/User/login` computed the OAuth redirect but the controller returned only `new { token = loginResult.Token }`.	The authorization code was discarded server-side. The caller received `{"token":null}` and the flow stalled with no error.
6	`TokenService` called `RSA.Create(2048)` inside its constructor, and the service was registered `Scoped`.	A brand-new signing keypair per HTTP request. Tokens were unverifiable by anyone — including Cerberus itself. No JWKS endpoint existed either.

> **ROOT CAUSE WORTH DWELLING ON**
>
> Defect 1 is the most instructive. `[FromQuery]` on a complex type binds each property by matching the query-string key to the *property name*, case-insensitively. `client_id` does not match `ClientId` — the underscore makes them different identifiers. Binding silently produced a default-initialised object rather than raising an error, so the failure surfaced as a misleading "*Unsupported response_type*" message that pointed at the wrong parameter entirely.

4. The complete end-to-end flow

The diagram below is the whole system: browser redirects, the credential exchange, the PKCE round trip, and the subsequent validation of the resulting token by a service that never spoke to the client. Dotted arrows are browser redirects; solid arrows are direct HTTP calls.

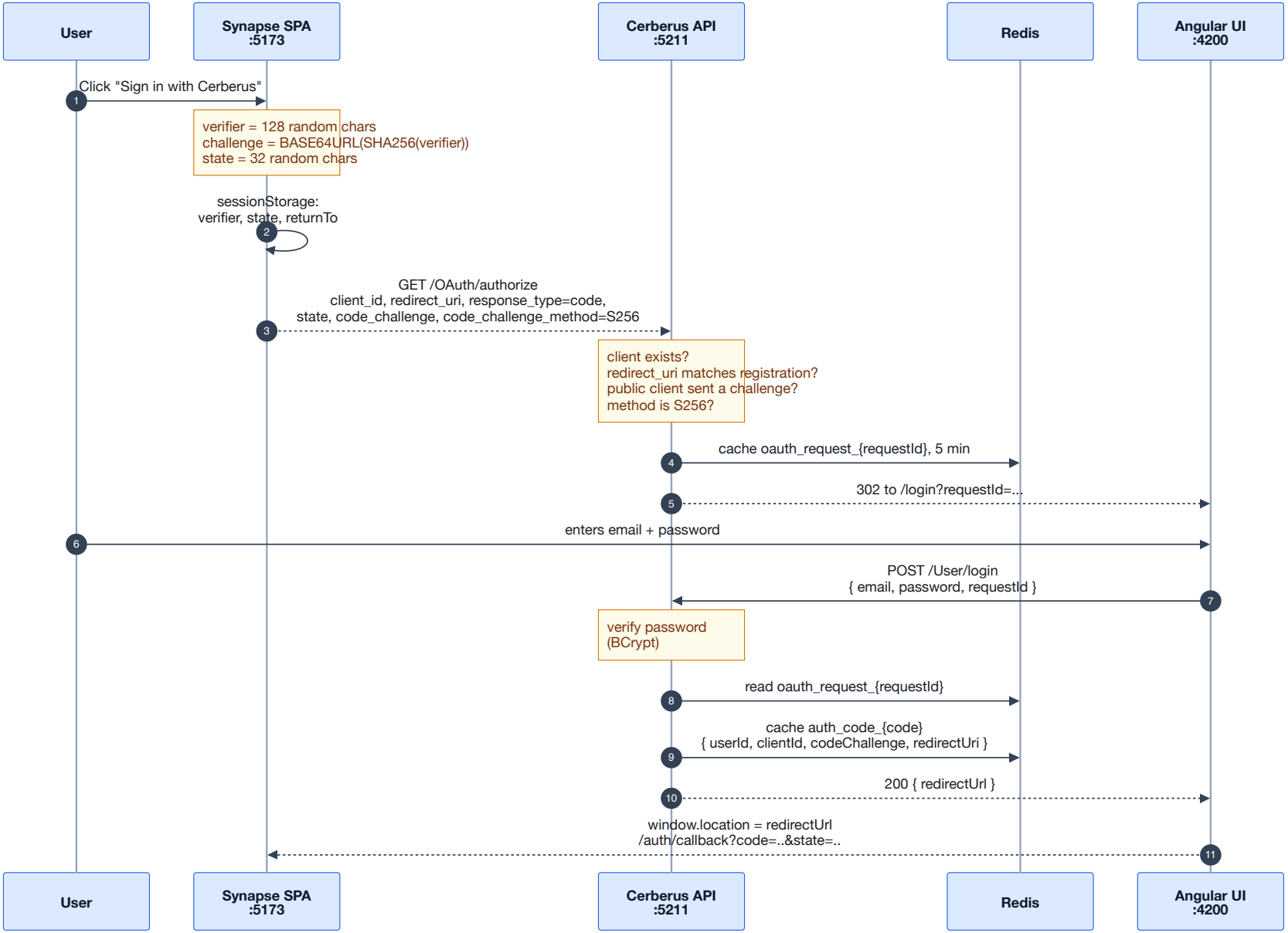

Figure 2a — Phase one: authorization request and credential collection. The SPA never sees the password; the Angular UI never sees the OAuth parameters.

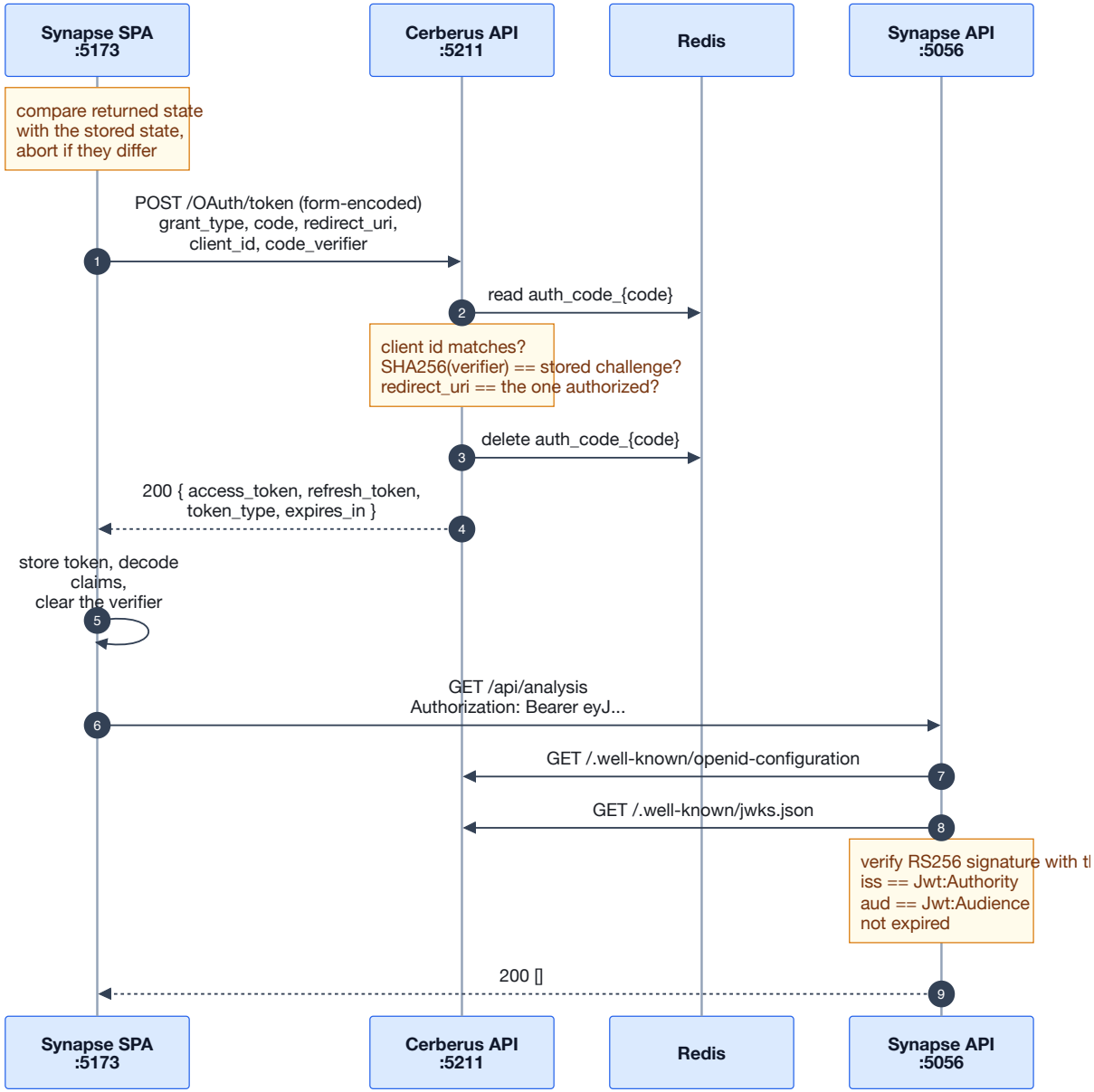

Figure 2b — Phase two: PKCE-protected code exchange, then resource access. The discovery and JWKS fetches happen once and are cached thereafter.

> **NOTE ON STEP 13**
>
> The `requestId` is the hinge of the whole design. Cerberus caches the pending authorization request server-side and hands the browser only an opaque identifier. The Angular UI therefore never needs to understand, carry or re-transmit OAuth parameters — it echoes one opaque string back, and the server reconstitutes the full request. This keeps `redirect_uri` and `code_challenge` out of reach of the login page entirely, so a compromised login page cannot alter where the code is delivered.

The MFA branch

When the account has MFA enabled the flow forks after password verification. The pending authorization request survives the detour because the `requestId` is stored *with the challenge*, server-side — the browser never has to hold it.

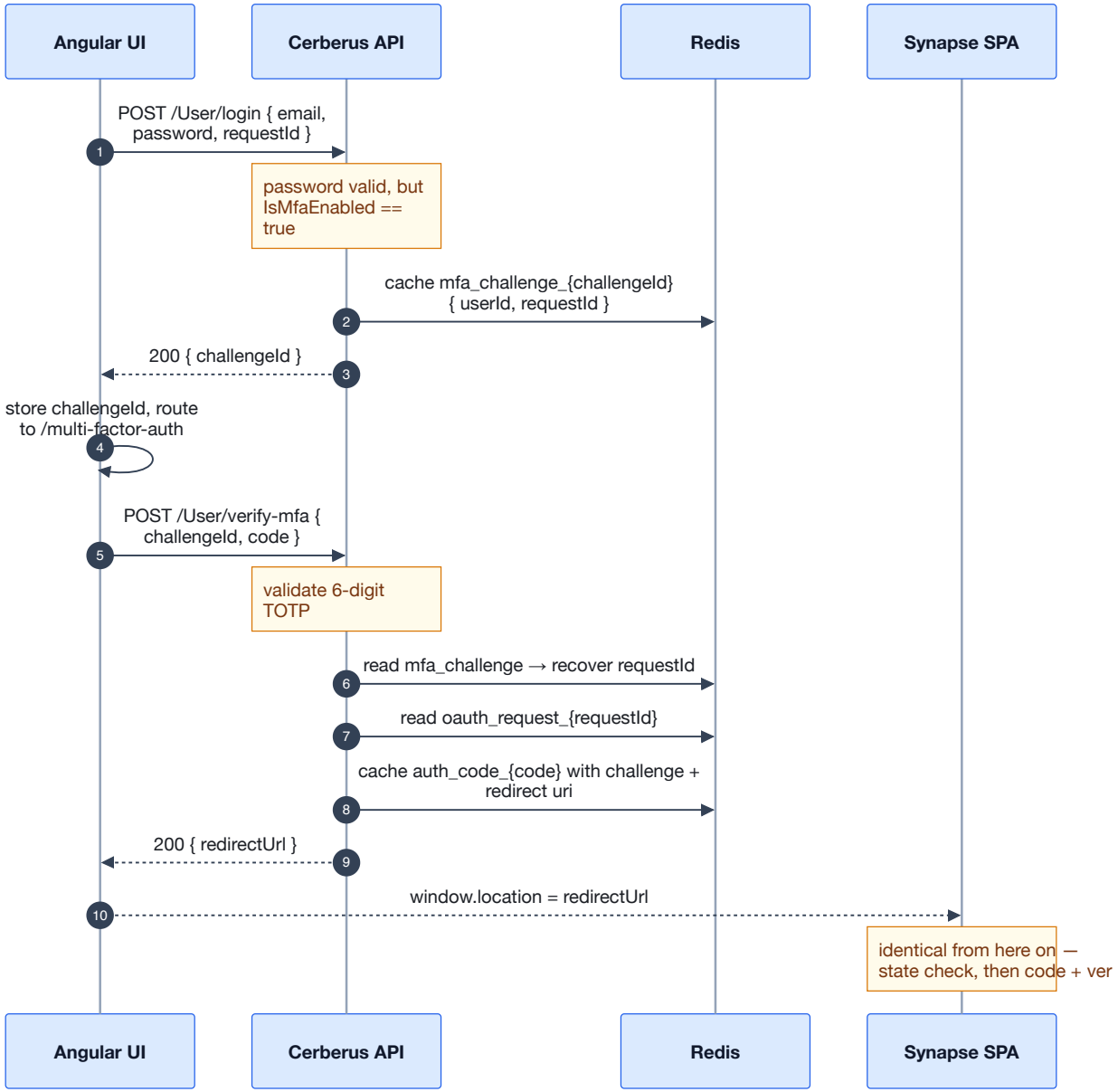

Figure 3 — The MFA detour. The OAuth context is carried entirely in server-side cache, keyed by the challenge.

5. Endpoint-by-endpoint changes

5.1 Parameter binding at the controller FIXED

Rather than decorating the application-layer DTOs with MVC attributes — which would have dragged `Microsoft.AspNetCore.Mvc` into a layer that deliberately depends only on `Domain` — each OAuth parameter is now bound explicitly at the controller and mapped into the DTO there.

```
public async Task<IActionResult> AuthorizeClientAsync(
    [FromQuery(Name = "client_id")]             string? clientId,
    [FromQuery(Name = "redirect_uri")]          string? redirectUri,
    [FromQuery(Name = "response_type")]         string? responseType,
    [FromQuery(Name = "state")]                 string? state,
    [FromQuery(Name = "scope")]                 string? scope,
    [FromQuery(Name = "code_challenge")]        string? codeChallenge,
    [FromQuery(Name = "code_challenge_method")] string? codeChallengeMethod,
    CancellationToken cancellationToken = default)
```

This has a secondary benefit beyond correctness: the OAuth wire contract is now visible at the endpoint signature. A reader can see exactly which spec parameters are supported without tracing through to a DTO in another project.

The token endpoint received the same treatment with `[FromForm(Name = …)]`, plus an explicit `[Consumes("application/x-www-form-urlencoded")]`. RFC 6749 requires the token endpoint to accept form encoding, not JSON.

5.2 The authorize endpoint FIXED

Three changes, in `AuthorizeClientCommandHandler`.

PKCE is now mandatory for public clients

```
var isPublicClient = string.IsNullOrEmpty(client.ClientSecret);

if (isPublicClient && string.IsNullOrWhiteSpace(authRequest.CodeChallenge))
{
    return Result.Failure<AuthorizeClientResultDTO>(
        "code_challenge is required for public clients");
}

if (!string.IsNullOrWhiteSpace(authRequest.CodeChallenge) &&
    !string.Equals(authRequest.CodeChallengeMethod, Pkce.S256, StringComparison.Ordinal))
{
    return Result.Failure<AuthorizeClientResultDTO>(
        "Unsupported code_challenge_method; only S256 is accepted");
}
```

Enforcing this at *authorize* time rather than only at token time is the important part. If a public client were permitted to start a flow without a challenge, the code it receives would be bearer-equivalent — anyone intercepting the redirect could redeem it. Rejecting the request up front means no such code is ever minted.

The redirect is absolute and points at the login UI

```
var separator = _settings.LoginUrl.Contains('?') ? '&' : '?';
var redirectUrl = $"{_settings.LoginUrl}{separator}requestId={requestId}";
```

`LoginUrl` comes from configuration (`OAuth:LoginUrl`) through a new `IOAuthSettings` abstraction. The application layer has no reference to `Microsoft.Extensions.Configuration`, so the interface is declared in `Application/Abstraction/Services` and implemented in `Infrastructure/Services/OAuthSettings.cs`, which reads configuration and fails fast at construction with an actionable message if it is missing.

5.3 The token endpoint FIXED

This handler changed the most. Four distinct issues.

A runtime bug in cache deserialisation

```
// before
var cached = await _cacheService.GetAsync<dynamic>($"auth_code_{req.Code}");
if (cached.ClientId != req.ClientId) { ... }
```

`CacheService` deserialises with `System.Text.Json`, so `dynamic` resolves to a `JsonElement`. A `JsonElement` has no `ClientId` member, so this line throws `RuntimeBinderException` at runtime rather than reading the stored value. Replaced with the concrete `AuthorizationCodeDTO`.

Branching on registration, verifying PKCE

```
if (isPublicClient)
{
    if (!Pkce.Verify(req.CodeVerifier, cached.CodeChallenge))
        return Result.Failure<TokenResponseDTO>("Invalid code_verifier");
}
else
{
    if (!_securityService.CheckSecret(req.ClientSecret ?? string.Empty, client.ClientSecret))
        return Result.Failure<TokenResponseDTO>("Invalid client credentials");

    // Confidential clients may still use PKCE; honour a registered challenge.
    if (!string.IsNullOrWhiteSpace(cached.CodeChallenge) &&
        !Pkce.Verify(req.CodeVerifier, cached.CodeChallenge))
        return Result.Failure<TokenResponseDTO>("Invalid code_verifier");
}
```

redirect_uri is re-validated

RFC 6749 §4.1.3 requires the `redirect_uri` presented at the token endpoint to match the one the code was issued for. The value is compared against the copy *stored with the code*, not against anything the caller resends:

```
if (!string.IsNullOrWhiteSpace(cached.RedirectUri) &&
    !string.Equals(cached.RedirectUri, req.RedirectUri, StringComparison.Ordinal))
{
    return Result.Failure<TokenResponseDTO>(
        "redirect_uri does not match the authorization request");
}
```

Richer claims

The handler now loads the user and passes name and email into token generation, so a relying party can render "signed in as…" without a second round trip to the user endpoint.

5.4 Client registration FIXED

Public clients are registered by supplying no secret. The original code hashed whatever it was given, and `HashSecret("")` produces a perfectly valid BCrypt hash — so an empty secret produced a client that *looked* confidential and was then required to present a secret it had never been issued.

```
ClientSecret = string.IsNullOrWhiteSpace(requestedClient.ClientSecret)
    ? string.Empty
    : _securityService.HashSecret(requestedClient.ClientSecret),
```

Registering the Synapse client:

```
curl -X POST http://localhost:5211/OAuth/clients \
  -H 'Content-Type: application/json' \
  -d '{"clientId":"synapse-spa","clientSecret":"",
       "redirectUri":"http://localhost:5173/auth/callback",
       "allowedScopes":"openid profile email"}'
```

> **OPERATIONAL NOTE**
>
> `redirectUri` is compared with an exact ordinal string match, both at authorize and at token time. A trailing slash, a different port, or `127.0.0.1` instead of `localhost` will be rejected. This strictness is correct — loose redirect matching is a classic open-redirect and token-theft vector — but it is the most common source of setup friction.

5.5 The login and MFA endpoints FIXED

Both endpoints computed the OAuth redirect correctly and then threw it away at the controller. `/User/login` returned only the token field:

```
// before: an OAuth login returned {"token": null}
return Ok(new { token = loginResult.Token });

// after
if (!string.IsNullOrEmpty(loginResult.RedirectUrl))
    return Ok(new { redirectUrl = loginResult.RedirectUrl });

return Ok(new { token = loginResult.Token });
```

`/User/verify-mfa` had a related but distinct problem: it returned `Ok(result)` — the `Result<LoginSecurityDTO>` wrapper itself. Callers received `{ value, isSuccess, isFailure, error }` instead of the payload, inconsistent with every other endpoint. It now mirrors the login shape.

6. The WellKnownController and token validation

What discovery is for

The Synapse API must decide whether a token it has never seen, issued by a service it does not share a secret with, is genuine. With RS256 that is done by signature verification: Cerberus signs with a private key nobody else holds, and any party can verify with the corresponding public key.

That leaves one problem — *how does the resource server obtain the public key, and how does it know which key signed a given token when keys rotate?* OpenID Connect Discovery answers this with two well-known endpoints, which is precisely what `WellKnownController` provides.

Endpoint	Purpose
`/.well-known/openid-configuration`	Metadata document. Declares the issuer, the endpoint URLs, supported grant types and challenge methods, and — critically — `jwks_uri`.
`/.well-known/jwks.json`	The JSON Web Key Set: the public half of the signing key, with a `kid` identifying it.

Configuring the resource server is then a single setting. `AddJwtBearer` takes `Authority`, fetches the discovery document, follows `jwks_uri`, caches the keys, and validates every token against them:

```
"Jwt": {
  "Authority": "http://localhost:5211",
  "Audience":  "synapse-spa"
}
```

> **WHY AUDIENCE IS THE CLIENT ID**
>
> Cerberus sets `audience: clientId` when building the token. The `aud` claim answers "who is this token *for*", and here it identifies the client the user authorized. So the Synapse API's `Jwt:Audience` must be `synapse-spa` — the client id — not a name for the API itself. This is a genuine point of confusion and was the cause of a real failure during integration: `appsettings.Development.json` was overriding it back to `synapse-api`, producing "*The audience 'synapse-spa' is invalid*".

The signing key

The discovery endpoints are only useful if the key is stable. The original `TokenService` generated a fresh keypair in its constructor and was registered `Scoped` — a new key per HTTP request. Every token was signed by a key that ceased to exist the moment the request completed.

Key resolution now follows a three-step precedence, and the service is a singleton:

1. `JWT:PrivateKeyPem` from configuration, for real deployments.
2. A PEM file at `JWT:SigningKeyPath`, if it exists.
3. Otherwise generate one *and persist it*, logging a warning. A fresh checkout works with no setup, and still survives a restart.

```
// Singleton, not scoped: the service owns the signing key, and resolving a new
// one per request is what produced a fresh keypair on every call.
services.AddSingleton<ITokenService, TokenService>();
```

The `kid` is derived from the key itself using the RFC 7638 JWK thumbprint — SHA-256 over a canonical JSON encoding of the public parameters, with members in a spec-mandated order and no whitespace:

```
var canonical = JsonSerializer.Serialize(new Dictionary<string, string>
{
    ["e"]   = Base64UrlEncoder.Encode(parameters.Exponent),
    ["kty"] = "RSA",
    ["n"]   = Base64UrlEncoder.Encode(parameters.Modulus)
});

return Base64UrlEncoder.Encode(SHA256.HashData(Encoding.UTF8.GetBytes(canonical)));
```

Deriving rather than assigning the identifier means the `kid` in a token header can never disagree with the `kid` published in the JWKS, because both are computed from the same key material.

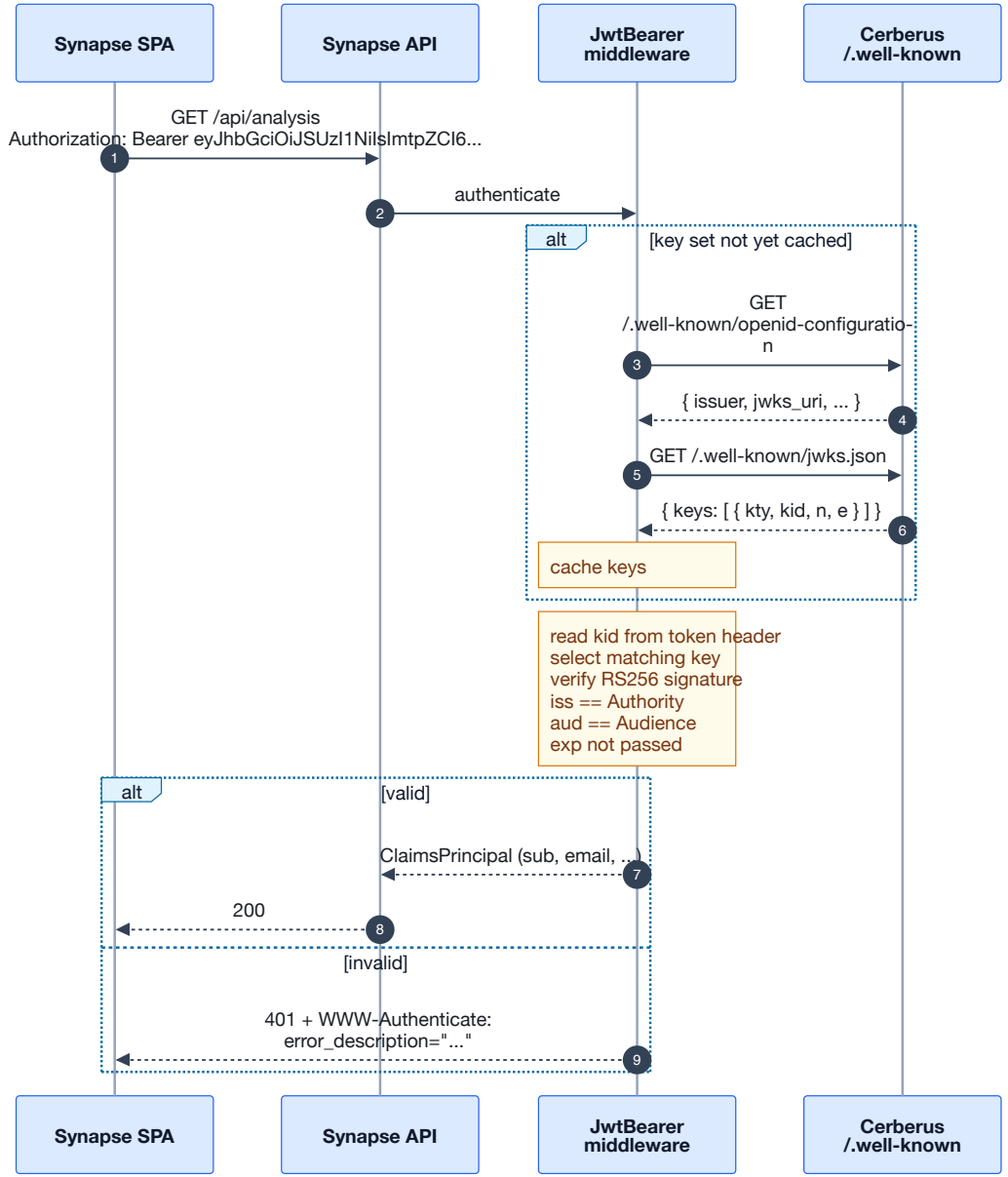

Figure 4 — Token validation. No shared secret and no call back to the client; trust flows entirely from the published key.

DEBUGGING TIP

When a token is rejected, the reason is in the `WWW-Authenticate` response header, not the body. `curl -D - …` and read it — it states plainly whether the failure was audience, issuer, signature or lifetime.

7. Browser-side authentication in the React client

Authentication in the Synapse client is split into small single-purpose modules under `src/features/auth/`, rather than one context object that does everything.

Module	Responsibility
`pkce.ts`	Verifier, challenge and state generation.
`tokenStorage.ts`	Persistence, JWT decoding, expiry checks.
`oauth.ts`	The protocol: authorize redirect, code exchange.
`AuthProvider.tsx`	React state, session restore, expiry timer.
`RequireAuth.tsx`	Route guard.
`sessionEvents.ts`	Decouples the axios interceptor from React.

Entropy and the state parameter

The verifier is drawn from `crypto.getRandomValues`. This is worth calling out because the Angular application's own `PkceService` uses `Math.random()`, which is not a cryptographic generator — its output is predictable from observed values, and a predictable verifier defeats PKCE entirely.

`state` is a separate random value serving a different purpose: CSRF protection on the callback. Without it, an attacker can feed a victim a callback URL carrying the attacker's own authorization code, causing the victim's session to be silently bound to the attacker's account. The client rejects any callback whose state does not match:

```
if (!expectedState || returnedState !== expectedState) {
  loginAttemptStorage.clear()
  throw new OAuthError(
    'The sign-in response could not be verified.',
    'The state parameter did not match the one sent with the request. …')
}
```

Where the tokens live

Access and refresh tokens go in `localStorage`; the PKCE verifier and state go in `sessionStorage`. That split is deliberate:

- Tokens must survive a page reload, or every refresh would bounce the user to the login screen.
- The verifier and state are single-use, scoped to one login attempt, and must not outlive the tab. They are cleared in a `finally` block so a failed exchange cannot leave a reusable verifier behind.

ACCEPTED TRADE-OFF

`localStorage` is readable by any script on the origin, so a successful XSS can exfiltrate the token. The mitigations that matter are keeping the token short-lived — Cerberus issues 15-minute tokens — and not introducing XSS, rather than attempting to hide the value from same-origin script, which is not achievable. The alternative is an `HttpOnly` cookie issued by a backend-for-frontend, which moves the code exchange server-side and is the stronger pattern if this ever leaves local development.

Session lifecycle

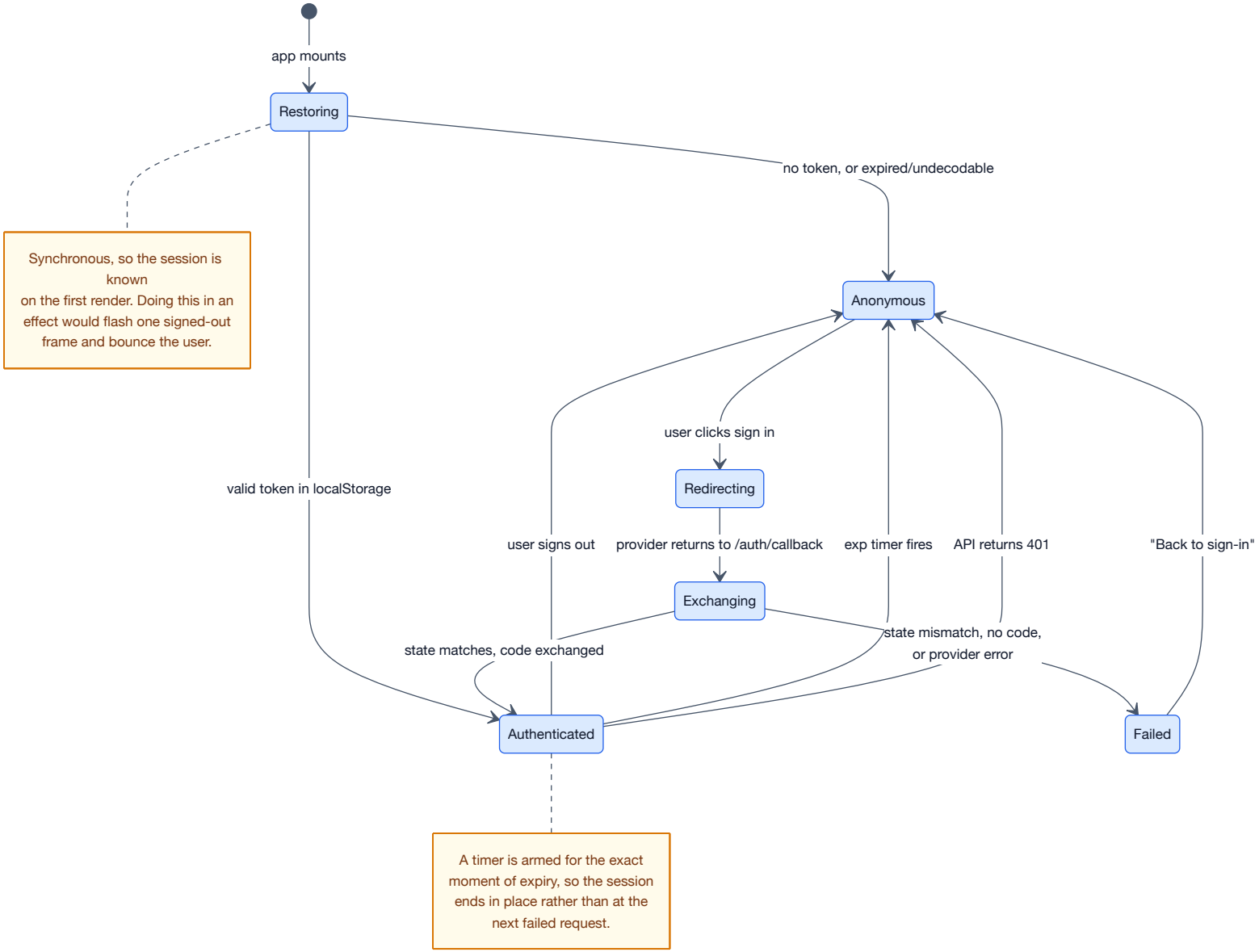

Figure 5 — Session state. Three independent paths lead back to Anonymous: explicit sign-out, local expiry, and server rejection.

Restoration must be synchronous

```
const [session, setSession] = useState<SessionState>(restoreSession)
```

Reading `localStorage` is synchronous, so the session is known on the very first render. Doing it in a `useEffect` instead would render one frame as signed-out — long enough for `RequireAuth` to redirect an already-authenticated user to the login page on every reload. React 19's `react-hooks/set-state-in-effect` lint rule flags exactly this pattern.

Expiring in place

```
const msRemaining = Math.max(claims.exp * 1000 - Date.now(), 0)
const timer = window.setTimeout(logout, msRemaining)
return () => window.clearTimeout(timer)
```

The clamp to zero matters: an already-expired token schedules an immediate timeout rather than calling `logout()` synchronously inside the effect, which would cascade an extra render.

Server-driven expiry

The axios response interceptor cannot import React state, and it is constructed before the component tree exists. A tiny event emitter bridges the gap:

```
if (error.response?.status === 401) {
  tokenStorage.clear()
  emitSessionExpired()   // AuthProvider subscribes and calls logout()
}
```

Only `401` clears the session. `403` deliberately does not — that means authenticated but not permitted, and signing the user out in response would be both wrong and confusing. The previous implementation checked for `status === 101`, which is the *Switching Protocols* informational code and can never appear here, so expired sessions were never handled at all.

Single-use codes and StrictMode

The callback page guards the exchange with a ref:

```
const hasStarted = useRef(false)
if (hasStarted.current) return
hasStarted.current = true
```

React StrictMode double-invokes effects in development. Without this guard the second invocation would attempt to redeem an authorization code the server has already consumed and deleted, producing a spurious *"Invalid or expired code"* failure on every development sign-in.

8. The Angular application as authorization UI

The Cerberus web application was written as a standalone login for itself: authenticate, store a token, navigate to the user menu. Acting as the authorization step of *another* application's OAuth flow is a different role, and it had none of the required wiring.

Gap	Consequence
Never read `requestId` from the query string.	The OAuth context was lost the moment the page loaded.
Never sent it with the login request.	The server took the non-OAuth branch and issued a session token instead of an authorization code.
Only handled `res.token` and `res.challengeId`.	On an OAuth login `token` is `null` by design, so the page silently did nothing. This was the visible symptom: a login that appeared to succeed and went nowhere.

The corrected response handling checks the OAuth outcome first:

```
if (!!res.redirectUrl) {
    // A full page navigation, not router.navigate: the target is a
    // different origin and Angular's router cannot leave the app.
    window.location.href = res.redirectUrl;
    return;
}
```

The same fix was applied to the MFA component, which had the identical defect one step later in the flow.

Two smaller corrections were made while in these files:

- The challenge was written to `localStorage` as `'Challange-Token'` but read by `challenge.guard.ts` as `'Challenge-Token'`. Latent only because the guard is currently stubbed to `return true`; it would have failed the moment the guard was enabled.
- `loginUser` sent `Authorization: Bearer null` on an anonymous endpoint, because it reused the authenticated-header helper before any token exists.

9. Cross-origin resource sharing

Three separate origins make browser requests in this system, so CORS is not incidental — it is load-bearing in three different places, each with different mechanics.

9.1 Why the browser is involved at all

The same-origin policy blocks a page on `localhost:5173` from reading a response from `localhost:5211`. Ports differ, so the origins differ. CORS is the server's mechanism for opting in.

The distinction that matters most for debugging: for a *simple* request the browser sends it and then withholds the response if the headers do not permit access — the server has already executed it. For anything else the

browser first sends a **preflight** `OPTIONS` and refuses to send the real request unless the preflight approves it. A JSON `POST` is preflighted; a form-encoded `POST` is not.

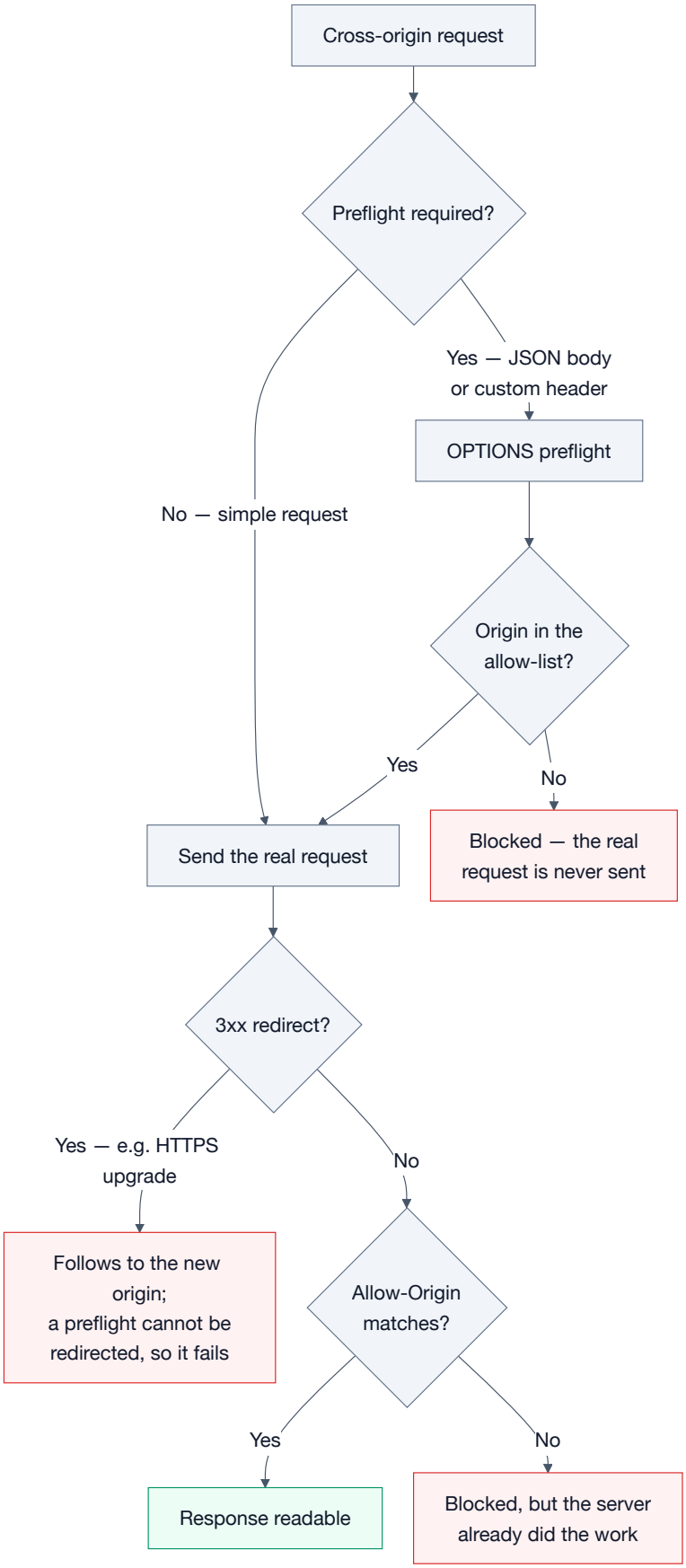

Figure 6 — Preflight decision path, and the redirect interaction that broke the integration.

9.2 Cerberus — from one hard-coded origin to configuration

The policy allowed exactly one origin, written into the source:

```
// before
corsPolicyBuilder.WithOrigins("http://localhost:4200")
```

The Angular app was the only client when that was written. The Synapse client calls `/OAuth/token` directly from `localhost:5173`, so it was blocked outright. Origins now come from configuration:

```
var allowedOrigins = builder.Configuration
    .GetSection("Cors:AllowedOrigins")
    .Get<string[]>() ?? ["http://localhost:4200"];

options.AddPolicy(allowSpecificOrigin, corsPolicyBuilder =>
{
    corsPolicyBuilder
        .WithOrigins(allowedOrigins)
        .AllowAnyHeader()
        .AllowAnyMethod()
        .AllowCredentials();
});
```

```
"Cors": {
  "AllowedOrigins": [ "http://localhost:4200", "http://localhost:5173" ]
}
```

> **WHY NOT ALLOWANYORIGIN**
>
> `AllowCredentials()` and `AllowAnyOrigin()` are mutually exclusive by specification — the browser rejects a wildcard `Access-Control-Allow-Origin` on a credentialed request, and ASP.NET Core throws at startup if both are configured. An explicit allow-list is required regardless, and is the correct posture for an authorization server: it bounds which applications can invoke the token endpoint from a browser.

9.3 The HTTPS redirect interaction KEY FINDING

Cerberus calls `app.UseHttpsRedirection()`. Run under its `https` launch profile it binds both 7077 and 5211, and then upgrades every plain-HTTP request with a 307. Observed directly during integration:

```
STEP 1  GET http://localhost:5211/OAuth/authorize
        -> 302 https://localhost:7077/OAuth/authorize?...   (not the login page)

STEP 4  POST http://localhost:5211/OAuth/token
        -> HTTP 307
```

This breaks a browser client in two compounding ways:

1. The upgraded target uses the ASP.NET development certificate, which the browser does not trust by default. The request fails at the TLS layer.
2. Where a preflight is involved, redirects are not permitted on the preflight itself, so the request is rejected before the real call is ever attempted.

Note also the middleware ordering: `UseCors` sits *before* `UseHttpsRedirection`, so a preflight is short-circuited with a valid 204 while the subsequent real request is still 307'd. The preflight succeeding makes the failure look unrelated to CORS, which is what makes it hard to diagnose.

The resolution for local development is to run the `http` profile only. With no HTTPS port bound there is nothing to redirect to, and the entire flow stays on one scheme. Every participant was aligned on `http://localhost:5211`, including the Angular application's `environment.apiUrl`, which had been pointing at `https://localhost:7077`.

9.4 Synapse API

The API uses a default policy naming the SPA origin, with the middleware ordered `UseCors` → `UseAuthentication` → `UseAuthorization`. That order is required: a preflight carries no `Authorization` header, so if authentication ran first every preflight would be rejected with a 401 and no cross-origin call would ever succeed.

```
"Cors": { "AllowedOrigins": [ "http://localhost:5173" ] }
```

This is also why the Vite dev server is pinned with `strictPort: true`. Vite's default is to move to 5174 if 5173 is occupied; that silent port change would produce CORS failures and a rejected OAuth redirect, both far harder to diagnose than a start-up error.

9.5 MinIO — a different mechanism entirely

The third origin is object storage. Because captures are uploaded from the browser straight to MinIO, MinIO must permit the SPA's origin. Two things make this unlike the other two cases:

- MinIO does not implement the S3 `PutBucketCors` API, so this cannot be configured through the SDK. It reads an environment variable: `MINIO_API_CORS_ALLOW_ORIGIN`.
- It is not enough for the request to succeed — the `ETag` response header must be *readable* by script. Multipart uploads are assembled from the ETag of each part, so if the header is not exposed cross-origin the upload cannot be completed even though every chunk transferred successfully.

The upload client detects precisely this and reports it, rather than failing opaquely:

```
const eTag = xhr.getResponseHeader('ETag')

if (!eTag) {
  reject(new UploadError(
    'The storage service did not return a readable ETag.',
    'The chunk uploaded, but its ETag header is not exposed to the browser, so the '
    + 'parts cannot be assembled. Start MinIO with MINIO_API_CORS_ALLOW_ORIGIN set '
    + "to this app's origin."))
}
```

10. Other logic worth noting

10.1 Bearer tokens over Server-Sent Events

The analysis stream is delivered with SSE, and the browser's `EventSource` API cannot set request headers — so it cannot send `Authorization`. The API opts into reading the token from the query string, narrowly:

```
OnMessageReceived = context =>
{
    if (string.IsNullOrEmpty(context.Token) &&
        context.Request.Path.StartsWithSegments("/api/analysis") &&
        context.Request.Query.TryGetValue("access_token", out var token))
    {
        context.Token = token;
    }

    return Task.CompletedTask;
}
```

The scoping is what makes this acceptable: it applies only when no header was supplied and only under `/api/analysis`. It remains a real trade-off — URLs are logged by servers and proxies far more readily than headers — and is mitigated in practice by the 15-minute token lifetime.

10.2 Claims mapping

`MapInboundClaims` is disabled, so the raw OIDC `sub` claim is not silently rewritten to the long-form .NET `NameIdentifier` URI. The extension that reads the caller's identity checks both, so either convention works:

```
var value =
    principal.FindFirstValue(JwtRegisteredClaimNames.Sub)
    ?? principal.FindFirstValue(ClaimTypes.NameIdentifier);
```

10.3 Just-in-time user provisioning

Cerberus owns the user record; the Synapse database only needs a row for its foreign keys. Rather than synchronising directories, the API provisions on first sight:

```
await _userRepository.EnsureAsync(userId, string.Empty, string.Empty, cancellationToken);
```

The identifier is the `sub` claim, so the two systems agree on identity without sharing a database.

10.4 Two storage clients, one signature

Presigned URLs are signed against a specific host — the host forms part of the signature. The storage adapter therefore holds two S3 clients: an internal one for the API's own operations and for URLs handed to the Zeek service, and a public one for URLs handed to the browser.

This produced a subtle failure. `MinIO:Endpoint` was `http://localhost:9000`, but that URL is also given to the Zeek container, where `localhost` is the container itself. The download could never succeed. Both sides

must agree on one name, so the endpoint is now `http://minio:9000` — resolved by Docker's internal DNS inside the network, and by a hosts entry on the development machine.

10.5 Error shape

The token endpoint returns the structure OAuth clients expect for a rejected grant (RFC 6749 §5.2) rather than a bare string:

```
return BadRequest(new
{
    error = "invalid_grant",
    error_description = result.Error
});
```

11. Security posture and remaining gaps

Verified as working correctly — each of these was exercised directly:

Check	Result
Wrong `code_verifier`	`400 invalid_grant`
Missing `code_verifier` on a public client	`400 invalid_grant`
Mismatched `redirect_uri` at token time	`400 invalid_grant`
Replay of an already-redeemed code	`400` — invalid or expired
Authorize without a challenge (public client)	`400` — challenge required
Garbage or absent bearer token at the API	`401`

Gaps that remain, in rough priority order:

1. **No refresh token grant.** `/OAuth/token` handles only `authorization_code`. A refresh token is issued but never persisted and cannot be redeemed, so sessions simply end after 15 minutes. Implementing this needs a storage and rotation decision, not just a code change.
2. **A failed exchange does not revoke the code.** The code is deleted on success and otherwise left to its 5-minute expiry. RFC 6819 recommends revoking on a failed verification attempt to remove the brute-force window.
3. **No `nonce` support and no ID token.** The discovery document advertises OIDC-shaped metadata, but only an OAuth access token is issued.
4. **Tokens in `localStorage`.** Discussed in §7 — acceptable here, worth revisiting behind a BFF for production.
5. **The Zeek service is unauthenticated and fetches arbitrary URLs.** It must remain on an internal network; exposing it provides a server-side request forgery primitive.
6. **The Angular client's own PKCE service uses `Math.random()`.** Not on the Synapse path, but it undermines PKCE for the Angular application's own flow and should be moved to `crypto.getRandomValues`.

12. Configuration reference

Every value below must agree across the four components. The issuer and authority in particular are compared as exact strings.

Setting	Location	Value
`JWT:Issuer`	Cerberus `appsettings.json`	`http://localhost:5211`
`OAuth:LoginUrl`	Cerberus `appsettings.json`	`http://localhost:4200/login`
`Cors:AllowedOrigins`	Cerberus `appsettings.json`	`[4200, 5173]`
`Jwt:Authority`	Synapse API	`http://localhost:5211` — must equal the issuer
`Jwt:Audience`	Synapse API	`synapse-spa` — the client id
`Cors:AllowedOrigins`	Synapse API	`http://localhost:5173`
`VITE_SSO_BASE_URL`	Synapse client `.env`	`http://localhost:5211`
`VITE_SSO_CLIENT_ID`	Synapse client `.env`	`synapse-spa`
`VITE_SSO_REDIRECT_URI`	Synapse client `.env`	`http://localhost:5173/auth/callback` — exact match required
`environment.apiUrl`	Angular	`http://localhost:5211`
`MINIO_API_CORS_ALLOW_ORIGIN`	MinIO container	`http://localhost:5173`

VERIFIED END TO END

The complete flow was exercised in a browser: sign-in from the Synapse client, redirect through Cerberus to the Angular login page carrying a `requestId`, credential submission, redirect back to the callback, code exchange, and an authorized API call. The resulting token carried `sub=ede06d10-…`, `preferred_username=Dana Analyst`, `aud=synapse-spa`, `iss=http://localhost:5211`; `GET /api/analysis` returned `200`, and the PKCE verifier was consumed and cleared.